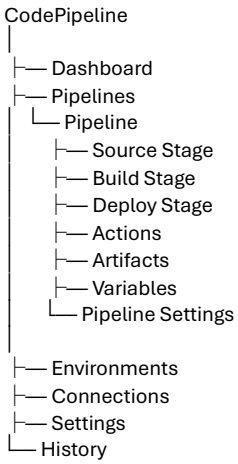
CodePipeline — Console Dashboard Wise Important Interview Questions & Answers



1. What is AWS CodePipeline?
Answer:
AWS CodePipeline is a CI/CD service used to automate the process of delivering application changes.
It connects different stages such as source, build, test, and deployment into one automated pipeline.
Whenever new code is committed, the pipeline can automatically start the required stages.
It helps developers deliver application changes faster and consistently.
For example:
Suppose a developer pushes code to GitHub.
CodePipeline can automatically get the code, send it to CodeBuild, and then deploy the new version to an EC2 or ECS environment.
This avoids doing each step manually.
Example:
Developer
↓
Source
↓
CodeBuild
↓
CodeDeploy
↓
Production
Simple ga gurthupettuko:
CodePipeline = Automate the complete CI/CD flow

---

2. What is a Pipeline in CodePipeline?
Answer:
A Pipeline is the complete workflow that defines how application changes move from source to deployment.
A pipeline normally contains multiple stages such as Source, Build, Test, and Deploy.
Each stage can contain one or more actions.
The pipeline executes these stages in the configured order.
For example:
Suppose we create a pipeline for a web application.
The pipeline first downloads code from the source, builds the application, and then deploys it to the production environment.
If one stage fails, the next stage normally does not continue.
Example:
Pipeline
↓
Source
↓
Build
↓
Test
↓
Deploy
Simple ga gurthupettuko:
Pipeline = Complete application delivery workflow

---

3. What is a Source Stage?
Answer:
The Source Stage is the first stage where CodePipeline gets the application source code.
It can connect with supported source providers such as CodeCommit, GitHub, or Amazon S3.

When a new source change is detected, it can start a new pipeline execution.
The source stage produces the input that is passed to the next stage.
For example:
Suppose our application code is stored in GitHub.
When a developer pushes new code, CodePipeline detects the change and starts the pipeline.
The source code is then passed to the build stage.
Example:
Developer
↓
GitHub
↓
Source Stage
↓
Build Stage
Simple ga gurthupettuko:
Source Stage = Where pipeline gets the code

---

4. What is a Build Stage?
Answer:
The Build Stage is used to compile, package, or test the application before deployment.
It can use services such as AWS CodeBuild to perform these tasks.
The build process can create an artifact that is passed to the deployment stage.
If the build fails, the pipeline can stop before deployment.
For example:
Suppose our application contains Java code.
CodePipeline sends the source code to CodeBuild, which compiles the code and creates a deployable package.
That package is then passed to the next stage.
Example:
Source Code
↓
Build Stage
↓
CodeBuild
↓
Application Artifact
Simple ga gurthupettuko:
Build Stage = Build and prepare application

---

5. What is a Deploy Stage?
Answer:
The Deploy Stage takes the application artifact and deploys it to the target environment.
It can integrate with services such as CodeDeploy, ECS, Lambda, or other supported deployment targets.
This stage normally runs after the source and build stages succeed.
It helps automate application releases to environments such as development or production.
For example:
Suppose CodeBuild creates a new application package.
The Deploy Stage can send that package to CodeDeploy, which installs it on the production EC2 instances.
The complete deployment happens automatically through the pipeline.
Example:
Build Artifact
↓
Deploy Stage
↓
CodeDeploy
↓
Production EC2
Simple ga gurthupettuko:
Deploy Stage = Release application to target environment

---

6. What is an Action in CodePipeline?
Answer:
An Action is an individual operation performed inside a pipeline stage.
For example, a Source Stage may have an action that retrieves code from GitHub.
A Build Stage may have an action that starts CodeBuild.
Multiple actions can exist inside a stage depending on the pipeline design.
For example:
Suppose the Build Stage uses CodeBuild.
The stage contains a build action that tells CodePipeline to start the CodeBuild project.
After that action succeeds, the pipeline can move to the next stage.
Example:
Build Stage

↓
Build Action
↓
CodeBuild
↓
Success

Simple ga gurthupettuko:
Action = Individual task inside a stage

---

7. What are Artifacts in CodePipeline?

Answer:
Artifacts are files produced by one pipeline stage and passed to another stage.
For example, the source stage can produce source files as an input artifact.
The build stage can process those files and produce a deployable output artifact.
These artifacts allow different stages to exchange application files.

For example:
Suppose CodeBuild creates app.zip after building the source code.
CodePipeline can pass that artifact to the deployment stage.
The deployment service then uses the artifact to release the application.

Example:
Source
↓
Source Artifact
↓
Build
↓
Build Artifact
↓
Deploy

Simple ga gurthupettuko:
Artifact = Files passed between pipeline stages

---

8. What are Pipeline Variables?

Answer:
Pipeline variables are values that can be used by actions and stages during a pipeline execution.
They can provide information such as branch names, commit IDs, or other configured values.
Variables help stages use dynamic information without hardcoding everything.
They are useful when designing flexible CI/CD pipelines.

For example:
Suppose the pipeline needs to know which Git branch triggered the execution.
A variable can provide the branch information to a later action.
The action can then use that value during the build or deployment process.

Example:
Pipeline
↓
Variables
↓
Branch / Commit Information
↓
Build / Deploy

Simple ga gurthupettuko:
Variables = Dynamic values used during pipeline execution

---

9. What is Pipeline History?

Answer:
Pipeline History shows previous executions of a pipeline.
It allows us to check whether previous executions succeeded, failed, or are still in progress.
We can open an execution to see individual stage and action results.
This is useful for troubleshooting and tracking application releases.

For example:
Suppose today's production deployment failed.
We can open Pipeline History and compare the failed execution with an earlier successful execution.
This can help identify which stage or action caused the problem.

Example:
Pipeline History
↓
Execution 1 → Success
Execution 2 → Success
Execution 3 → Failed
↓
Check Stage / Action

Simple ga gurthupettuko:
History = Previous pipeline executions

---

10. What are Connections in CodePipeline?
Answer:
Connections allow CodePipeline to securely connect with supported external source or third-party providers.
For example, a connection can be used to connect CodePipeline with GitHub.
The connection provides an authorized way for the pipeline to access the external repository.
This avoids manually handling credentials inside the pipeline.
For example:
Suppose our source code is stored in GitHub.
We create a CodePipeline connection to GitHub and select the required repository and branch in the Source Stage.
CodePipeline can then retrieve source changes through that connection.
Example:
GitHub
↓
CodePipeline Connection
↓
Source Stage
↓
Pipeline
Simple ga gurthupettuko:
Connection = Secure connection between CodePipeline and external provider

---

11. What is Pipeline Settings?
Answer:
Pipeline Settings contain configuration options that control how the pipeline operates.
Depending on the pipeline type, these settings can include pipeline execution behavior and other pipeline-level configurations.
They help control how new executions are handled when another execution is already running.
We should review these settings when designing production pipelines.
For example:
Suppose a developer pushes several changes while a production deployment is already running.
Pipeline execution settings determine how those new executions are handled.
This helps prevent unwanted or conflicting deployments.
Example:
New Code Change
↓
Pipeline Settings
↓
Execution Behavior
↓
Pipeline Execution
Simple ga gurthupettuko:
Pipeline Settings = Control pipeline behavior

---

12. What is the CodePipeline Dashboard used for?
Answer:
The CodePipeline Dashboard provides an overview of pipelines and their current execution status.
From the console, we can see whether pipelines are running, succeeded, or failed.
We can open a pipeline to inspect its stages, actions, and execution details.
It is mainly used to monitor and manage the CI/CD workflow.
For example:
Suppose a developer pushes new code to the repository.
We can open the CodePipeline dashboard and check whether Source, Build, and Deploy stages completed successfully.
If a stage fails, we can open that stage and investigate the error.
Example:
CodePipeline Dashboard
↓
Pipeline
↓
Source → Build → Deploy
↓
Success / Failed
↓
Check Details
Simple ga gurthupettuko:
Dashboard = Monitor the complete CI/CD pipeline
ChatGPT is AI and can make mistakes.